부록 · 파이썬으로 DB 다루기

PyMySQL — 넣기 · 읽기 · 고치기 · 지우기

2-5 참고의 「기본 5가지」 다음 단계입니다. 연습용 표 py_memo 를 만들어 네 가지 동작을 한 번씩 해 봅니다. 수업용 표(products 등)는 건드리지 않습니다.

준비

```bash
# 2-1 의 de-mysql(또는 2-3 의 de-mongo)이 켜져 있어야 합니다
docker start de-mysql

# 가상환경을 켜고 필요한 것만 설치합니다
cd ~/data-eng/week2 && source .venv/bin/activate
pip install pymysql sqlalchemy pandas python-dotenv pymongo
python3 -c "import sys; print(sys.prefix)"     # .venv 로 끝나야 가상환경에 설치된 것
```

코드

```python
# py_mysql_crud.py — PyMySQL 로 넣기·읽기·고치기·지우기 (2-1 의 de-mysql · shop DB)
import pymysql

conn = pymysql.connect(host='127.0.0.1', port=3306, user='root', password='dataeng123',
                       database='shop', charset='utf8mb4',
                       cursorclass=pymysql.cursors.DictCursor)

with conn:                                      # 블록이 끝나면 연결을 닫는다
    with conn.cursor() as cur:
        # 연습용 표 — 몇 번 돌려도 같은 결과가 나오게 지우고 새로 만든다
        cur.execute("DROP TABLE IF EXISTS py_memo")
        cur.execute("""
            CREATE TABLE py_memo (
              id    INT AUTO_INCREMENT PRIMARY KEY,
              title VARCHAR(100) NOT NULL,
              done  TINYINT NOT NULL DEFAULT 0
            )""")

        # 넣기 — lastrowid 로 방금 생긴 id 를 받는다
        cur.execute("INSERT INTO py_memo (title) VALUES (%s)", ('MySQL 복습',))
        print('새 id:', cur.lastrowid)
        cur.executemany("INSERT INTO py_memo (title) VALUES (%s)", [('pandas 읽기',), ('MongoDB
적재',)])
        conn.commit()

        # 읽기
        cur.execute("SELECT id, title, done FROM py_memo ORDER BY id")
        for row in cur.fetchall():
            print(row)

        # 고치기 — execute 가 돌려주는 수가 바뀐 행 수
        n = cur.execute("UPDATE py_memo SET done = 1 WHERE title = %s", ('MySQL 복습',))
        conn.commit()
        print('고친 행:', n)

        # 지우기
        n = cur.execute("DELETE FROM py_memo WHERE done = %s", (1,))
        conn.commit()
        print('지운 행:', n)

        cur.execute("SELECT COUNT(*) AS n FROM py_memo")
        print('남은 행:', cur.fetchone()['n'])
```

```bash
python3 py_mysql_crud.py
```

실행 결과 (실측)

```text
새 id: 1
{'id': 1, 'title': 'MySQL 복습', 'done': 0}
{'id': 2, 'title': 'pandas 읽기', 'done': 0}
{'id': 3, 'title': 'MongoDB 적재', 'done': 0}
고친 행: 1
지운 행: 1
남은 행: 2
```

짚을 점

- cursorclass=DictCursor — 결과가 딕셔너리라 row['title'] 처럼 칸 이름으로 꺼냅니다. 빼면 튜플이라 row[1] 처럼 순서로 꺼내야 합니다.
- cur.lastrowid — 방금 넣은 행의 AUTO_INCREMENT 번호입니다. 주문을 넣고 그 번호로 주문 상세를 넣을 때 씁니다.
- cur.execute(...) 가 돌려주는 수 — 영향을 받은 행 수입니다. UPDATE·DELETE 뒤에 0 이면 조건에 맞는 행이 없었던 것입니다.
- with conn: — 블록이 끝나면 연결을 닫습니다. 닫는 것을 잊어 연결이 쌓이는 일을 막습니다.

commit 을 빠뜨리면

INSERT 뒤에 commit 없이 연결을 닫으면 넣은 행이 사라집니다. 같은 연결 안에서는 보이기 때문에 알아채기 어렵습니다 (실측: 같은 연결에서 3건 → 다시 연결하면 2건).

그래서 「넣었는데 다른 창에서 안 보인다」면 commit 부터 의심합니다. 바꾸는 명령(INSERT·UPDATE·DELETE) 뒤에는 conn.commit() 을 붙이는 습관을 들입니다.

부록 · 파이썬으로 DB 다루기

값은 %s 자리로, 비밀번호는 .env 로

코드가 돌아가는 것만큼 중요한 두 가지 습관입니다. 실제로 막아지는 것을 눈으로 확인합니다.

왜 문자열로 이어 붙이면 안 되나 — SQL 인젝션

검색어를 f-string 으로 SQL 에 바로 이어 붙이면, 검색어 안의 따옴표가 SQL 의 일부가 됩니다. 아래 코드는 누군가 검색창에 「' OR '1'='1」을 넣은 경우입니다.

.env — 비밀번호를 코드 밖으로

코드와 같은 폴더에 .env 파일을 만들고 접속 정보를 적습니다. 코드에는 비밀번호가 남지 않습니다.

```env
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=dataeng123
DB_NAME=shop
```

```bash
# .env 는 깃에 올리지 않습니다 — .gitignore 에 한 줄
echo ".env" >> .gitignore
```

코드

```python
# py_mysql_safe.py — 값은 %s 자리로, 비밀번호는 .env 로
import os
import pymysql
from dotenv import load_dotenv

load_dotenv()      # 같은 폴더의 .env 를 읽어 환경변수로 올린다
conn = pymysql.connect(host=os.getenv('DB_HOST', '127.0.0.1'), port=int(os.getenv('DB_PORT',
'3306')),
                       user=os.getenv('DB_USER'), password=os.getenv('DB_PASSWORD'),
                       database=os.getenv('DB_NAME'), charset='utf8mb4')

keyword = "' OR '1'='1"     # 검색창에 누군가 이렇게 넣었다고 하자

with conn:
    with conn.cursor() as cur:
        # 나쁜 예 — 문자열로 이어 붙이면 조건이 무너져 전부 나온다
        cur.execute(f"SELECT name FROM products WHERE name = '{keyword}'")
        print('이어 붙인 경우:', len(cur.fetchall()), '행')

        # 좋은 예 — %s 자리로 넘기면 값 전체를 글자 하나로 다룬다
        cur.execute("SELECT name FROM products WHERE name = %s", (keyword,))
        print('%s 로 넘긴 경우:', len(cur.fetchall()), '행')
```

```bash
python3 py_mysql_safe.py
```

실행 결과 (실측)

```text
이어 붙인 경우: 5 행
%s 로 넘긴 경우: 0 행
```

이어 붙인 쪽은 WHERE name = '' OR '1'='1' 이 되어 조건이 항상 참이 됩니다. 그래서 상품 5개가 전부 나왔습니다. %s 로 넘긴 쪽은 그 글자 그대로의 이름을 찾았고, 그런 상품은 없으니 0 행입니다.

짚을 점

- %s 는 PyMySQL 의 자리 표시입니다. 숫자여도 %d 가 아니라 %s 를 씁니다. 값은 언제나 두 번째 인자(튜플)로 넘깁니다.
- 값이 하나일 때도 튜플로 씁니다 — (keyword,) 처럼 쉼표가 있어야 튜플입니다.
- 표 이름·칸 이름은 %s 로 넘길 수 없습니다(값 자리 전용). 이름을 사용자 입력으로 바꿔야 한다면 허용 목록에서 고르게 합니다.
- 수업처럼 root 로 접속하는 것은 실습용입니다. 실제 서비스는 필요한 권한만 준 계정을 따로 만들어 씁니다(부록 「DB 관리 명령어」의 사용자 만들기).

부록 · 파이썬으로 DB 다루기

pandas 로 읽고 · 가공하고 · 다시 넣기

3주차 pandas 로 바로 이어집니다. SQL 결과를 DataFrame 으로 받아 가공하고, 결과를 다시 표로 넣습니다. 연결은 SQLAlchemy 엔진으로 합니다.

코드

```python
# py_pandas_sql.py — pandas 로 표를 읽고, 가공해서, 다시 넣기
import pandas as pd
from sqlalchemy import create_engine, text

engine = create_engine('mysql+pymysql://root:dataeng123@127.0.0.1:3306/shop?charset=utf8mb4')

# 읽기 — SQL 결과가 바로 DataFrame. 값은 :이름 자리로 넘긴다
df = pd.read_sql(text("SELECT name, category, price, stock FROM products WHERE price >= :p"),
                 engine, params={'p': 15000})
print(df)

# 가공 — 분류별 상품 수와 재고 금액
df['stock_value'] = df['price'] * df['stock']
summary = (df.groupby('category', as_index=False)
             .agg(items=('name', 'count'), stock_value=('stock_value', 'sum')))
print(summary)

# 넣기 — if_exists='replace' 는 표를 지우고 새로 만든다('append' 는 뒤에 덧붙인다)
summary.to_sql('py_category_summary', engine, if_exists='replace', index=False)
print(pd.read_sql(text('SELECT * FROM py_category_summary'), engine))

# 큰 표는 나눠서 — chunksize 행씩 DataFrame 이 차례로 나온다
for part in pd.read_sql(text('SELECT id, name FROM products ORDER BY id'), engine, chunksize=
2):
    print('조각', len(part), '행:', list(part['id']))
```

```bash
python3 py_pandas_sql.py
```

실행 결과 (실측)

```text
      name category  price  stock
0  노트북 파우치       잡화  15000     40
1   무선 마우스       전자  28000     25
2  기계식 키보드       전자  89000     10
3      텀블러       잡화  19000     60
4  모니터 받침대       전자  32000     15
  category  items  stock_value
0       잡화      2      1740000
1       전자      3      2070000
  category  items  stock_value
0       잡화      2      1740000
1       전자      3      2070000
조각 2 행: [1, 2]
조각 2 행: [3, 4]
조각 1 행: [5]
```

짚을 점

- 엔진 주소 — mysql+pymysql://계정:비밀번호@호스트:포트/DB?charset=utf8mb4. 앞의 mysql+pymysql 은 「MySQL 에 PyMySQL 로 붙는다」는 뜻입니다.
- text("… :p") + params={'p': 값} — SQLAlchemy 에서는 자리 표시가 :이름 입니다. PyMySQL 의 %s 와 다릅니다. 값은 여기서도 자리로 넘깁니다.
- to_sql 의 if_exists — 'replace' 는 표를 지우고 새로 만들고, 'append' 는 뒤에 덧붙이고, 기본값 'fail' 은 표가 있으면 오류를 냅니다. index=False 를 빼면 DataFrame 의 번호가 index 라는 칸으로 들어갑니다.
- chunksize — 표가 커서 한 번에 메모리에 올리기 어려울 때 그 행 수만큼씩 나눠 받습니다. 결과가 DataFrame 하나가 아니라 차례로 나오는 조각들이 됩니다.

한글이 ë… ¸íŠ¸ë¶ 처럼 보이면

파이썬 코드가 아니라 넣을 때 이미 깨진 것입니다. 터미널에서 docker exec … mysql 로 한글을 넣을 때 문자 설정이 빠지면, 한글이 다른 글자 묶음으로 저장됩니다. 그러면 무엇으로 읽어도 깨져 보입니다(실측 — 이 문서를 준비하며 실제로 겪었습니다).

```bash
# 2-1 에서 쓴 접속 — -e LANG=C.UTF-8 이 그 문자 설정입니다
docker exec -it -e LANG=C.UTF-8 de-mysql mysql -uroot -pdataeng123 shop
```

이미 깨져 들어간 행은 고쳐지지 않습니다. 지우고 다시 넣습니다. 파이썬에서 넣는 경우에는 연결에 charset='utf8mb4'(엔진 주소는 ?charset=utf8mb4)를 꼭 붙입니다.

부록 · 파이썬으로 DB 다루기

SQLAlchemy 트랜잭션 — 다 되거나, 하나도 안 되거나

재고를 빼고 출고 기록을 남기는 두 동작은 함께 되거나 함께 안 돼야 합니다. 재고만 빠지고 기록이 없으면 숫자가 맞지 않습니다. 연습용 표 두 개로 확인합니다.

코드

```python
# py_sqlalchemy_tx.py — SQLAlchemy 로 묶음(트랜잭션) 처리: 다 되거나, 하나도 안 되거나
from sqlalchemy import create_engine, text

engine = create_engine('mysql+pymysql://root:dataeng123@127.0.0.1:3306/shop?charset=utf8mb4')

# 연습용 표 두 개 — 재고와 출고 기록 (shop 의 products 는 건드리지 않는다)
with engine.begin() as conn:
    conn.execute(text("DROP TABLE IF EXISTS py_stock"))
    conn.execute(text("DROP TABLE IF EXISTS py_stock_log"))
    conn.execute(text("CREATE TABLE py_stock (id INT PRIMARY KEY, stock INT NOT NULL)"))
    conn.execute(text("CREATE TABLE py_stock_log (id INT, delta INT)"))
    conn.execute(text("INSERT INTO py_stock VALUES (1, 40)"))

def ship(item_id, qty, fail=False):
    """재고를 빼고 기록을 남긴다 — begin() 블록은 끝까지 가면 commit, 예외가 나면 rollback"""
    try:
        with engine.begin() as conn:
            conn.execute(text("UPDATE py_stock SET stock = stock - :q WHERE id = :id"), {'q':
qty, 'id': item_id})
            if fail:
                raise RuntimeError('기록 저장 실패(일부러)')
            conn.execute(text("INSERT INTO py_stock_log VALUES (:id, :d)"), {'id': item_id,
'd': -qty})
        print(f'출고 {qty}개 완료')
    except RuntimeError as e:
        print('되돌림:', e)

ship(1, 5)
ship(1, 5, fail=True)

with engine.connect() as conn:
    stock = conn.execute(text("SELECT stock FROM py_stock WHERE id = 1")).scalar()
    logs = conn.execute(text("SELECT COUNT(*) FROM py_stock_log")).scalar()
print('재고', stock, '/ 기록', logs, '건')
```

```bash
python3 py_sqlalchemy_tx.py
```

실행 결과 (실측)

```text
출고 5개 완료
되돌림: 기록 저장 실패(일부러)
재고 35 / 기록 1 건
```

두 번 출고를 시도했고 두 번째는 일부러 실패시켰습니다. 재고가 30 이 아니라 35 인 것은, 두 번째 시도에서 이미 뺀 재고 5개가 통째로 되돌아갔기 때문입니다. 기록도 1건뿐입니다.

짚을 점

- engine.begin() — 블록이 끝까지 가면 commit, 블록 안에서 예외가 나면 rollback 을 알아서 합니다. PyMySQL 에서 손으로 하던 commit·rollback 을 묶어 준 것입니다.
- engine.connect() — 읽기만 할 때 씁니다. 여기서 바꾸는 명령을 쳤다면 conn.commit() 을 따로 해야 남습니다.
- .scalar() — 결과의 첫 행 첫 칸 하나만 꺼냅니다. COUNT(*) 같은 값 하나짜리 질의에 씁니다.
- CREATE·DROP 같은 표 만들기 명령은 MySQL 에서 되돌려지지 않습니다(바로 확정). 트랜잭션은 넣기·고치기·지우기를 묶는 것입니다.

부록 · 파이썬으로 DB 다루기

pymongo — 찾고 · 세고 · 묶고 · 고치기

2-3 에서 load_logs.py 로 넣은 접속 로그 1,000건(logdb.access_logs)을 파이썬에서 다룹니다. 먼저 2-3 의 de-mongo 를 켜고 load_logs.py 를 한 번 돌려 두세요.

준비

```bash
docker start de-mongo
docker exec de-mongo mongosh --quiet --eval "db.runCommand({ ping: 1 }).ok"   # 1 이면 준비 완료
cd ~/data-eng/week2 && source .venv/bin/activate
python3 load_logs.py          # 2-3 에서 만든 파일 — 몇 번 돌려도 같은 1,000건
```

코드

```python
# py_mongo.py — pymongo 로 찾고·세고·묶고·고치기 (2-3 의 de-mongo · logdb.access_logs)
from pymongo import MongoClient, ASCENDING, DESCENDING
import pandas as pd

client = MongoClient('mongodb://localhost:27017', serverSelectionTimeoutMS=5000)
col = client['logdb']['access_logs']

# 찾기 — 조건 · 가져올 칸(1) · 정렬 · 개수
for doc in (col.find({'level': 'ERROR'}, {'_id': 0, 'ts': 1, 'path': 1, 'error.code': 1})
               .sort('ts', ASCENDING).limit(3)):
    print(doc)

# 세기
print('ERROR', col.count_documents({'level': 'ERROR'}), '건')

# 묶기(aggregate) — 경로별 ERROR 수, 많은 순
pipeline = [
    {'$match': {'level': 'ERROR'}},
    {'$group': {'_id': '$path', 'n': {'$sum': 1}}},
    {'$sort': {'n': -1, '_id': 1}},
]
for row in col.aggregate(pipeline):
    print(row)

# 고치기 — 결제 경로 오류에 확인 표시 달기
r = col.update_many({'level': 'ERROR', 'path': '/api/pay'}, {'$set': {'checked': True}})
print('맞은', r.matched_count, '/ 바뀐', r.modified_count)

# 색인 — 자주 찾는 칸에 만든다. 이미 있으면 그대로 두고 이름만 돌려준다
print('색인:', col.create_index([('level', ASCENDING), ('ts', DESCENDING)]))

# pandas 로 — 커서를 리스트로 받아 DataFrame
df = pd.DataFrame(list(col.find({'source': 'load_logs'}, {'_id': 0, 'level': 1, 'status':
1})))
print(df.groupby('level').size())
```

```bash
python3 py_mongo.py
```

실행 결과 (실측)

```text
{'ts': datetime.datetime(2026, 8, 31, 9, 0, 9), 'path': '/', 'error': {'code': 'E64'}}
{'ts': datetime.datetime(2026, 8, 31, 9, 0, 18), 'path': '/', 'error': {'code': 'E35'}}
{'ts': datetime.datetime(2026, 8, 31, 9, 0, 21), 'path': '/api/pay', 'error': {'code': 'E67'}}
ERROR 201 건
{'_id': '/api/pay', 'n': 46}
{'_id': '/mypage', 'n': 42}
{'_id': '/search', 'n': 42}
{'_id': '/login', 'n': 40}
{'_id': '/', 'n': 31}
맞은 46 / 바뀐 46
색인: level_1_ts_-1
level
ERROR    201
INFO     586
WARN     213
dtype: int64
```

두 번째로 돌리면 「맞은 46 / 바뀐 0」이 나옵니다. 이미 checked 가 True 라 바꿀 것이 없었다는 뜻이고, 정상입니다(실측). 나머지 줄은 같습니다.

짚을 점

- find(조건, 가져올 칸) — 가져올 칸에 1 을 적은 것만 옵니다. _id 는 0 으로 적어야 빠집니다. 'error.code' 처럼 점으로 안쪽 칸을 고릅니다.
- count_documents — 조건에 맞는 문서 수입니다. 조건 없이 전부 세려면 {} 를 넘깁니다.
- aggregate — $match(거르기) → $group(묶기) → $sort(정렬)를 차례로 통과시킵니다. SQL 의 WHERE → GROUP BY → ORDER BY 와 같습니다. 같은 수가 있으면 순서가 흔들리지 않게 '_id' 로 한 번 더 정렬했습니다.
- update_many 의 결과 — matched_count 는 조건에 맞은 수, modified_count 는 실제로 바뀐 수입니다. 둘이 다르면 이미 그 값이었던 문서가 있다는 뜻입니다.
- serverSelectionTimeoutMS=5000 — 컨테이너가 꺼져 있을 때 기본 30초 대신 5초 만에 실패합니다. 부록 「실습 트러블슈팅」의 시간 초과 항목과 같은 상황입니다.
